

IVF 参数优化器 — 使用说明文档

概述

本工具用于在 **FAISS / IVF-PQ 向量检索实验**中，自动分析并寻找最优参数组合。
通过读取包含实验结果的 CSV 文件（例如 `ivf_gallery_query_metrics.csv`），
程序会根据用户定义的内置的多目标权重体系，计算每个配置的综合得分并排序，帮助快速评估以下指标：

- 构建时间： `build_time_s`
- 查询延迟： `query_time_ms`
- 索引体积： `index_size_mb`
- 向量召回率： `vector_recall_at_k`
- 标签精度： `label_precision_at_k_micro`

最终输出：

- 每种优化倾向（速度优先 / 内存优先 / 精度优先）的 **Top-N 参数组合表格**
- 每个组合的归一化指标与综合得分

环境依赖

```
pip install pandas numpy scikit-learn
```

快速使用

```
from ivf_optimizer import optimize_with_presets

results = optimize_with_presets(
    "ivf_gallery_query_metrics.csv",
    param_cols=["pca_dim", "nlist", "nprobe"],
    normalize="minmax",      # 可选 "minmax" 或 "zscore"
    filters=None,           # 例如 {"k": [20]} 可筛选实验
    top_n_each=3            # 每种倾向取前 3 个结果
)
```

运行后将会：

1. 自动载入数据；
2. 依次根据三种预设策略计算得分；
3. 输出每种策略下得分最高的前三个配置（并以交互表格形式展示）。

内置三种优化倾向（Presets）

名称	目标说明	主要权重分配
speed_first	注重响应速度，最小化查询与构建时间	<code>query_time_ms</code> 0.5 · <code>build_time_s</code> 0.3 · 其他 0.1~0.05
memory_first	注重内存占用，最小化索引体积	<code>index_size_mb</code> 0.6 · <code>build_time_s</code> 0.15 · <code>query_time_ms</code> 0.15
quality_first	注重检索精度与召回率	<code>label_precision_at_k_micro</code> 0.45 · <code>vector_recall_at_k</code> 0.35 · 其他 0.1~0.05

每个倾向的结果都会控制在控制台打印前 3 名参数组合（含得分），同时弹出交互表格以供比较。

算法原理

1. 归一化处理
 - 默认使用 *min-max* 标准化，将不同量纲的指标映射至 [0,1]。
 - 若指标方向为“最小化”，则取反 (1 - x) 使其转化为“越大越好”。
2. 加权求和得分
 - 各指标归一化后按设定权重线性组合，得到综合得分：
[
$$\text{score} = \frac{\sum_i w_i \cdot \text{norm}_i}{\sum_i w_i}$$

]
3. 排序与展示
 - 按得分从高到低排序，取前 `top_n_each` 行；
 - 输出结果表格包括参数列、原始值、归一化值与最终得分。

高级用法：自定义预设

你可以自定义优化倾向（权重与方向），传入 `presets` 参数覆盖默认设置：

```
presets = {
  "speed_first": {
    "query_time_ms": ("min", 0.5),
    "build_time_s": ("min", 0.3),
    "index_size_mb": ("min", 0.1),
    "label_precision_at_k_micro": ("max", 0.05),
    "vector_recall_at_k": ("max", 0.05),
  },
  "memory_first": {
    "index_size_mb": ("min", 0.6),
    "build_time_s": ("min", 0.15),
    "query_time_ms": ("min", 0.15),
    "label_precision_at_k_micro": ("max", 0.05),
    "vector_recall_at_k": ("max", 0.05),
  },
}
```

```

        "quality_first": {
            "label_precision_at_k_micro": ("max", 0.45),
            "vector_recall_at_k": ("max", 0.35),
            "query_time_ms": ("min", 0.10),
            "index_size_mb": ("min", 0.05),
            "build_time_s": ("min", 0.05),
        },
    },

results = optimize_with_presets(
    "ivf_gallery_query_metrics.csv",
    param_cols=["pca_dim", "nlist", "nprobe"],
    presets=presets
)

```

💡 实践建议

- 若 **k = ground truth 样本数**（例如都为 20），则 `recall` 与 `precision` 可视为等价指标，只保留其一即可。
- 若指标数量较多，建议权重总和约为 1.0，便于结果直观。
- 可切换 `normalize="zscore"`，用于减少极端值影响。

📊 输出结果解释

每个表格包含：

字段	含义
<code>pca_dim</code> , <code>nlist</code> , <code>nprobe</code>	关键参数组合
<code>build_time_s</code> , <code>query_time_ms</code> , <code>index_size_mb</code>	时间及体积指标
<code>label_precision_at_k_micro</code> , <code>vector_recall_at_k</code>	检索准确性指标
<code>norm_*</code>	归一化后指标
<code>score</code>	综合加权得分（越高越优）

🏠 输出示例

```

=== speed_first ===
   pca_dim  nlist  nprobe  score
0         0    128        1  0.846
1         0    256        1  0.843
2         0    512        1  0.838

```